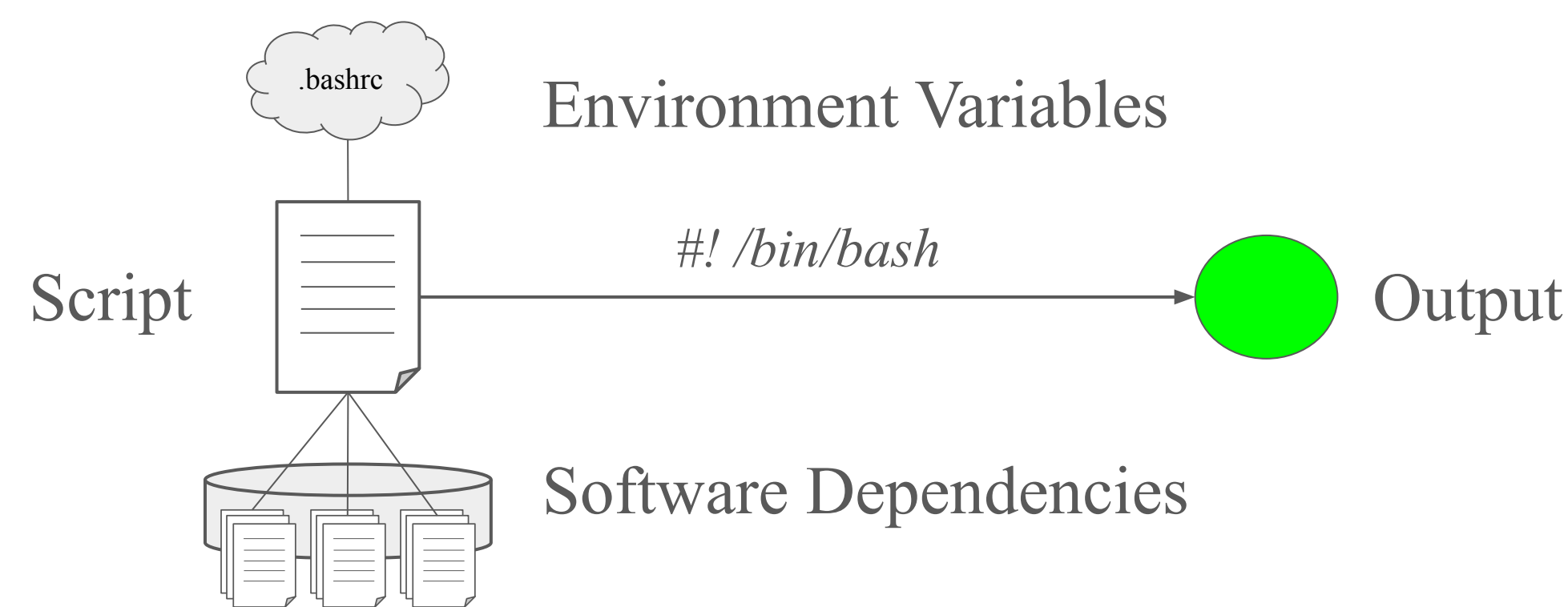


Contour: I/O Contracts to Reproduce and Distribute Scientific Applications

Colin Thomas¹, Andrés Iglesias², Douglas Thain¹

University of Notre Dame¹, Universidad Católica Santa María La Antigua²

Introduction: Scientific applications are run in a broad variety of cluster ecosystems, which use several layers of supporting technologies to connect users with data and manage resources. The applications themselves are composed with some number of highly specific software packages.

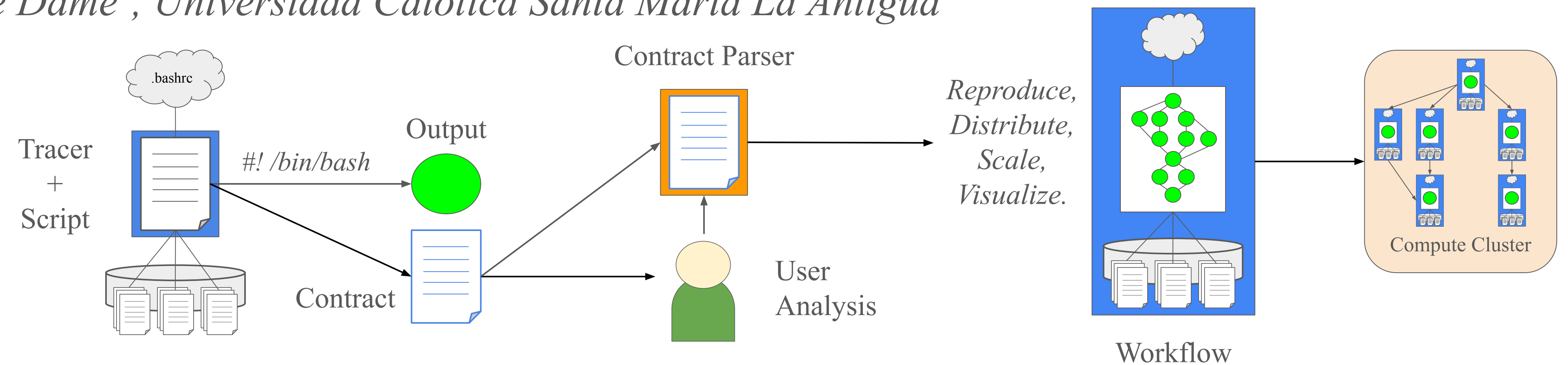


Many applications benefit in performance and versatility when described and executed as a task-based workflow, with order of execution defined by a DAG (Directed-Acyclic-Graph). The challenge in writing such a workflow is the description of tasks and their dependencies in order to form the DAG. Scientific software packages often exhibit implicit behavior and depend on unexpected information.

This work presents **Contour**: an *I/O analysis and workflow composition framework*, which observes a running application to create an execution contract. The contract describes the behavior of the application and facilitates construction of a high-level distributed workflow, enabling the scalability and performance benefits of a distributed execution.

Acknowledgment:

This work was supported in part by NSF award CNS-2317556



Tracing: *Contour* begins with an application to study/distribute. The application is run locally, and system calls are captured using a specialized strace invocation wrapped in the **Tracer**. The output of strace is parsed and formed into a **Contract**, describing the I/O behavior of the application process tree.

Some important process attributes are:

- Opened files and permissions.
- Files read and written to.
- Files executed. Arguments on invocation.
- Environment attributes.

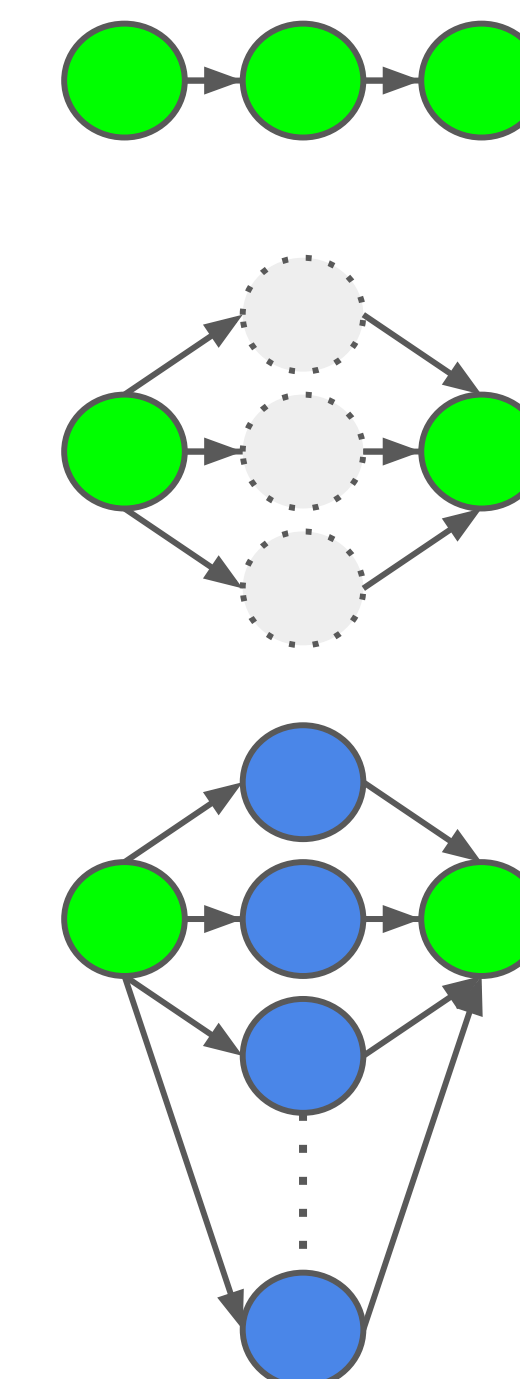
A process will produce such an entry in the contract:

```
pid 123:
X: /Montage/bin/mDiffFit
A: -t imglst aK_asky_04.fits
R: ak_asky_04.fits, imglst
W: dF_asky_04.fits
```

The contract can be used to study the behavior of an application, such as the implicitly named file written above. This implicit behavior is a key challenge of porting/developing scientific workflows. Applications will create files with implicit names, read and write to files in the same directory without suggestion, and make other assumptions naming and its surroundings. It would be impossible to identify all data dependencies by only reading the application script.

Workflow Generation: The contract is intended to be complete in describing an application's requirements in order to enable running the application as elsewhere, which could be another user's shell or as a workflow on remote cluster nodes.

To create the workflow template, the **Contract Parser** produces an expression of the application interpretable by the **Makeflow** workflow engine. It is likely a user would intend to scale an application from the version which was traced locally. To facilitate this, the application can be expressed highly abstracted to scale up concurrent jobs and substitute larger data sets.



```
rules : {
  {x: 1.sh, w:f},
  {x: 2.sh, r:f, w:y}
  {x: 3.sh, r:y, w:z}
}
```

How do we scale?

```
rules : {$A,$B,$C}
define : {
  B : template(
    'x:2_{v}.sh, r:f, w:y',
    for v in 100),
  A : ...,
```